

Ansible Concepts

Inventory

Since the whole point of Ansible is to configure and manage remote servers, you need to provide a list of your remote servers somehow. This is accomplished by adding these to a hosts file that could be configured globally for all your projects, or locally for a single project. Ansible calls this your inventory.

The inventory file is simply a text file conventionally named `hosts`, that has a list of all your servers. These servers can be grouped together to apply the same configuration to all servers in a group. You can even have groups of groups. This file can also include additional variables for configuring each host, such as the remote username, the kind of connection to use, the port, etc. It's even possible to create groups of groups and group-level variables and a lot more.

If you use a cloud hosting service like DigitalOcean or RackSpace or Amazon EC2, you can even use scripts that create a dynamic inventory based your hosting account.



Ansible Tower is the commercial version of Ansible that includes a web-based interface.

Here is an example of what this file could look like:

```
[databases]
```

```
db1.server.com ansible_user=ubuntu
```

```
db2.server.com ansible_host=192.168.1.27
```

```
[appservers]
```

```
app1.server.com:27
```

```
app2.server.com
```

Commands

With Ansible, rather than connecting to your server using SSH and then running commands, you specify a host on which to run a command and the command to run there. Ansible will connect to the server using SSH and run the task for you.

This may seem like a nuisance rather than a benefit, but remember that with Ansible you can have groups of hosts. This way you can run a command on hundreds of hosts at once. For instance if you have all your `webservers` defined under a group called `webservers` you can restart the Nginx webserver on all of them at once using:

```
ansible webservers -m service -a "name=nginx state=restarted"
```

In the above, `webservers` is the name of the group of hosts (or single host), and `-m service` here tells Ansible to run the `service` module used to restart services using `init`, `systemd` or `upstart` and following `-a` are the parameters to pass to the `service` module, which are the name of the service and its state.

If your command needs `sudo` to run, you can add the parameter `--become`, and add `--ask-become-pass` to have Ansible ask you the `sudo` password.

Playbooks

Running a command at a time has its uses, but what Ansible really is for, is building a list of such commands as tasks, and then applying these sequence of commands to a host to get it into the state you want.

Ansible does this using Playbooks; these are text files written using the YAML format that describe a servers configuration. In a playbook you specify a set of tasks to perform on each host or host group. When you run Ansible on this file, it will perform these tasks one-by-one on all the supplied hosts.

Tasks

A task is a single step in configuring a host. Such as creating a user, copying a file to the server, enabling / disabling / restarting a service, cloning / updating a code repository, installing / uninstalling / updating a package etc. Through multiple steps you can get the server set up the way you want.

Ansible only performs a task if its necessary. If the package you want is already present, it won't install it again. It won't copy over a configuration file if it hasn't changed, etc.

Tasks can also be grouped by tags. You can then provide a tag while using the Ansible command, to run only the tasks having that tag.

Notification Hooks

What if you only want to run a task based on whether another task needed to run? For instance, restart Apache only if its configuration file was updated. For this, Ansible has notifications and handlers.

Notification-handler tasks are only run if they are notified to run as the result of a standard task. Such a notification task could then be to restart Apache server. The Apache config setup task can then add the Apache server restart task as its notification task.

You can also use Ansible's integrations with SMS services and mobile push notification services to send a notification message based on what happens during a run of your playbook.

Roles

To simplify reuse, you can even bundle up a couple of tasks together into a role, and then apply one or more roles to a host. For instance you would create an `apache` role that does all the work of installing, configuring and starting Apache, and includes a hook to restart Apache if needed. This role can then be applied to a server.

You could also create a `database` role for your database, and have your web and database server on the same host by applying both roles to the same host, or separate them by applying the roles to different host.

Templates and Variables

It is essential for a configuration management tool to, well, manage configuration files. Often this means editing and changing bits in configuration files to fill in details like IP addresses, server ports,